SISTEMAS OPERACIONAIS

Exclusão mútua: espera ocupada

Prof. Matheus A. Souza
Eng. Software/Sist. Informação
PUC Minas

Princípios da boa exclusão mútua

1. Somente um processo pode estar na região crítica a cada momento

2. Não se pode fazer suposições sobre velocidade e ordem de execução de processos

3. Nenhum processo fora da região crítica pode impedir outros processos de entrarem em suas regiões críticas

4. Um processo não pode esperar indefinidamente para entrar em sua região crítica

Espera ocupada (*busy waiting*)

- Sem ajuda ou interferência do SO
  - Não há estado de bloqueio nem chamadas de sistema

- Processo que não pode adentrar a região crítica gasta o tempo de processador inutilmente até o fim do *quantum*

Espera ocupada (*busy waiting*)

> Alternativa 1: desabilitar interrupções

> Processo desabilita interrupções ao entrar e habilita ao sair da região crítica

Espera ocupada (*busy waiting*)

- Alternativa 1: desabilitar interrupções

- Processo desabilita interrupções ao entrar e habilita ao sair da região crítica

  - Espera indefinida
  - Segurança comprometida

Espera ocupada (*busy waiting*)

- Alternativa 1: desabilitar interrupções

- Processo desabilita interrupções ao entrar e habilita ao sair da região crítica

  - Espera indefinida
  - Segurança comprometida

- Pode ser usado pelo próprio SO

Espera ocupada (*busy waiting*)

> Alternativa 2: variável de bloqueio

> Usar variável *booleana* para indicar permissão de entrada

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

Espera ocupada (*busy waiting*)

> Alternativa 2: variável de bloqueio

> Seria ótimo, porém ***não funciona!!!***

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

Variável de bloqueio

```
Debitar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo -=v;
        c.setSaldo(saldo);
        lock = false;
```

```
Creditar(Conta c, valor v, lock)
    while(lock);
        lock = true;
        saldo = c.saldo();
        saldo +=v;
        c.setSaldo(saldo);
        lock = false;
```

O PROBLEMA SAIU DE *SALDO* E FOI PARA *LOCK*

Espera ocupada (*busy waiting*)

> Alternativa 3: alternância estrita

> Variável controla quem tem o direito de executar naquele ***turno***

> Funciona, pois a ação é de ceder a vez, não de tomar

Alternância estrita

```
P0(Conta c, valor v, turno)
   while(turno!=0);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
      turno = 1;
```

```
P1(Conta c, valor v, turno)
   while(turno!=1);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
      turno = 0;
```

Alternância estrita

```
P0(Conta c, valor v, turno)
   while(turno!=0);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
      turno = 1;
```

```
P1(Conta c, valor v, turno)
   while(turno!=1);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
      turno = 0;
```

turno
0

Alternância estrita

```
P0(Conta c, valor v, turno)
   while(turno!=0);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
      turno = 1;
```

```
P1(Conta c, valor v, turno)
   while(turno!=1);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
      turno = 0;
```

turno
0

Alternância estrita

```
P0(Conta c, valor v, turno)
   while(turno!=0);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
      turno = 1;
```

```
P1(Conta c, valor v, turno)
   while(turno!=1);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
      turno = 0;
```

turno
0

Alternância estrita

```
P0(Conta c, valor v, turno)
   while(turno!=0);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
      turno = 1;
```

```
P1(Conta c, valor v, turno)
   while(turno!=1);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
      turno = 0;
```

turno
1

Alternância estrita

```
P0(Conta c, valor v, turno)
   while(turno!=0);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
      turno = 1;
```

```
P1(Conta c, valor v, turno)
   while(turno!=1);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
      turno = 0;
```

turno
1

Alternância estrita

```
P0(Conta c, valor v, turno)
   while(turno!=0);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
      turno = 1;
```

```
P1(Conta c, valor v, turno)
   while(turno!=1);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
      turno = 0;
```

turno
1

Alternância estrita

```
P0(Conta c, valor v, turno)
   while(turno!=0);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
      turno = 1;
```

```
P1(Conta c, valor v, turno)
   while(turno!=1);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
      turno = 0;
```

turno
0

Espera ocupada (*busy waiting*)

- Alternativa 3: alternância estrita

- Funciona, porém...
  - Múltiplos processos e desempenho
  - Bloqueio desnecessário

SISTEMAS OPERACIONAIS

Exclusão mútua: Semáforos
Produtores e Consumidores

Prof. Matheus A. Souza
Eng. Software/Sist. Informação
PUC Minas

Sleep / wakeup

- Chamadas de sistema para que o processo peça a ida ao bloqueio (e o desbloqueio de outros, ao sair da RC)

- *Sleep* por dependência

- *Sleep* por período

Produtores e consumidores

- Modelo clássico de CIP

- Duas classes de processos
  - Produtores de itens (escrevem na memória)
  - Consumidores de itens (retiram da memória)

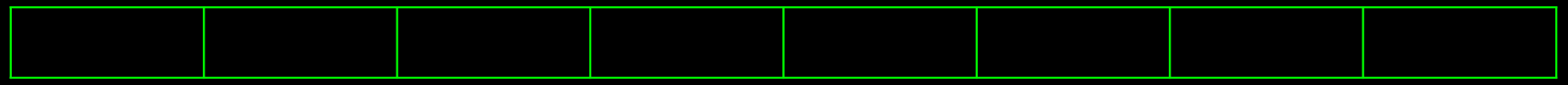
- *Buffer* compartilhado

Produtores e consumidores

- Modelo clássico de CIP

- Duas classes de processos
  - Produtores de itens (escrevem na memória)
  - Consumidores de itens (retiram da memória)

- *Buffer* compartilhado

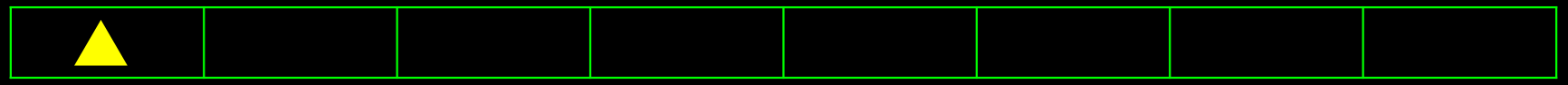


Produtores e consumidores

- Modelo clássico de CIP

- Duas classes de processos
  - Produtores de itens (escrevem na memória)
  - Consumidores de itens (retiram da memória)

- *Buffer* compartilhado

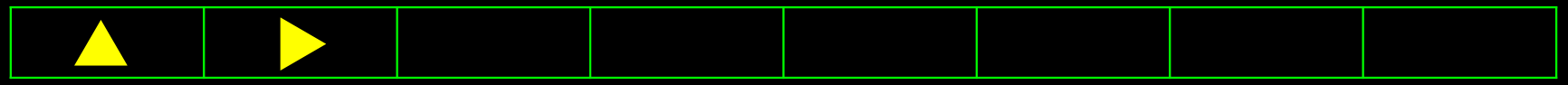


Produtores e consumidores

- Modelo clássico de CIP

- Duas classes de processos
  - Produtores de itens (escrevem na memória)
  - Consumidores de itens (retiram da memória)

- *Buffer* compartilhado

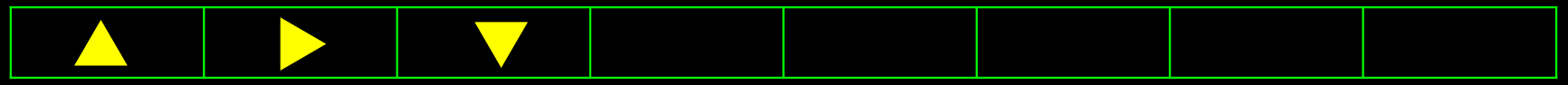


Produtores e consumidores

- Modelo clássico de CIP

- Duas classes de processos
  - Produtores de itens (escrevem na memória)
  - Consumidores de itens (retiram da memória)

- *Buffer* compartilhado

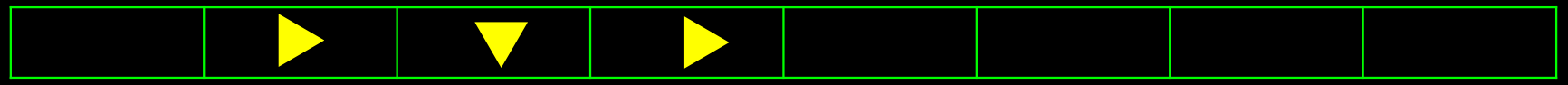


Produtores e consumidores

- Modelo clássico de CIP

- Duas classes de processos
  - Produtores de itens (escrevem na memória)
  - Consumidores de itens (retiram da memória)

- *Buffer* compartilhado

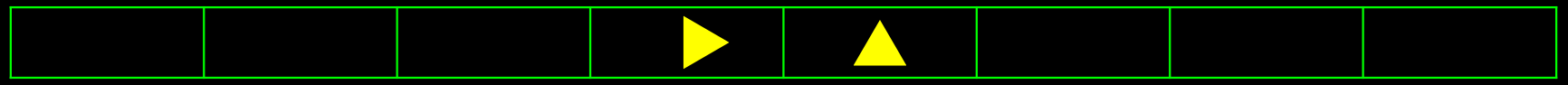


Produtores e consumidores

- Modelo clássico de CIP

- Duas classes de processos
  - Produtores de itens (escrevem na memória)
  - Consumidores de itens (retiram da memória)

- *Buffer* compartilhado

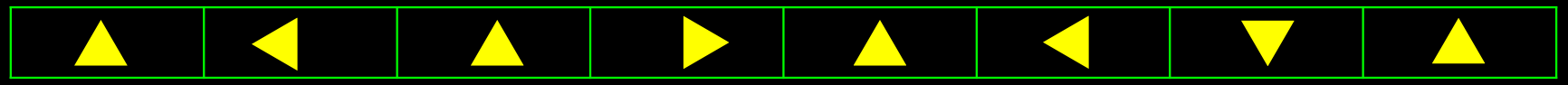


Produtores e consumidores

- Modelo clássico de CIP

- Duas classes de processos
  - Produtores de itens (escrevem na memória)
  - Consumidores de itens (retiram da memória)

- *Buffer* compartilhado



Produtores e consumidores

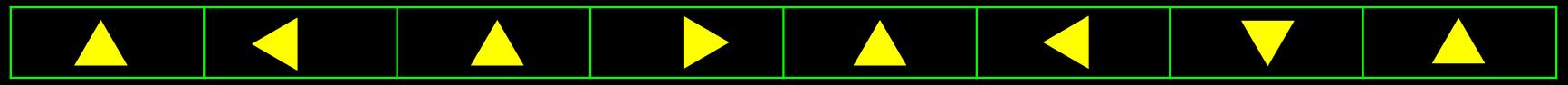
```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      if(cont==N) sleep(cont);
      inserir(it);
      cont++;
      if(cont==1) wakeup(cont);
   }
}
```

```
Consumidor(){
   Item meu;
   while(true){
      if(cont==0) sleep(cont);
      meu = retirar();
      cont--;
      if(cont==N-1) wakeup(cont);
      consumir(meu);
   }
}
```

Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```

Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```

Produtores e consumidores

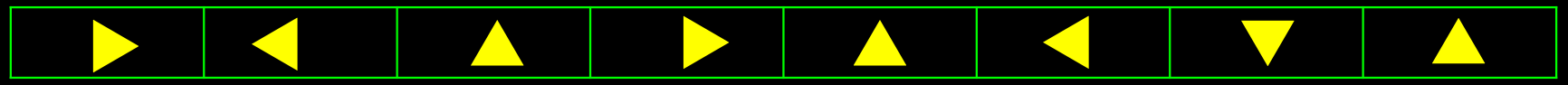
```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```

Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

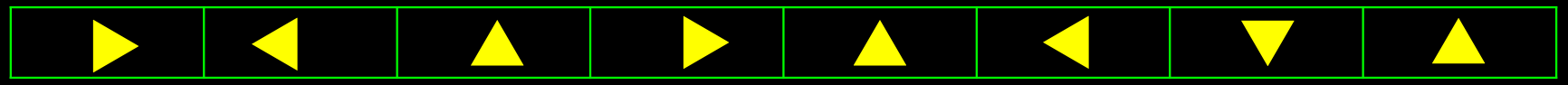
```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

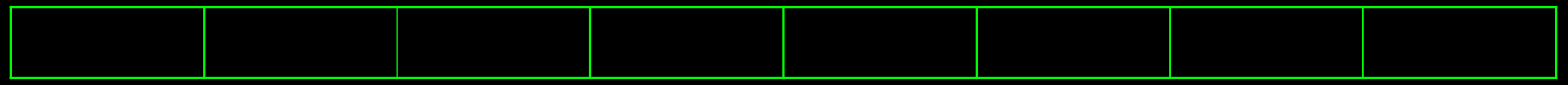
```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

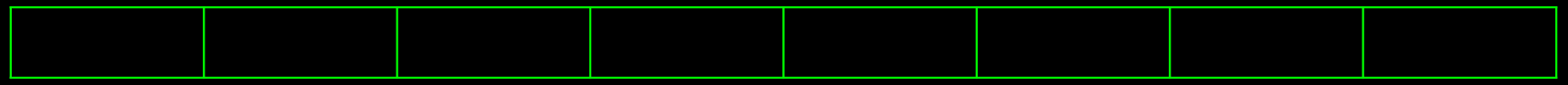
```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont—;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

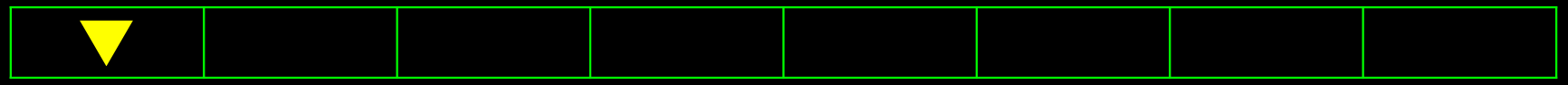
```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

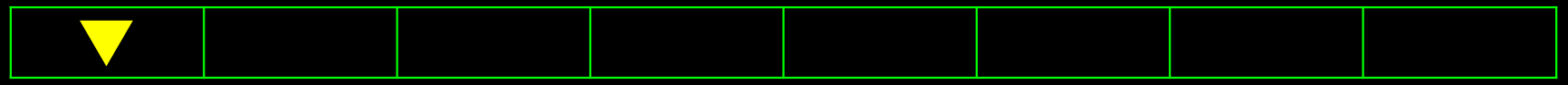
```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

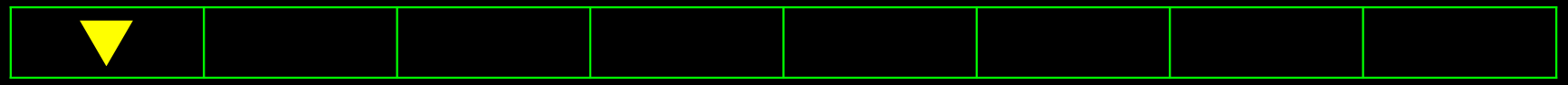
```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

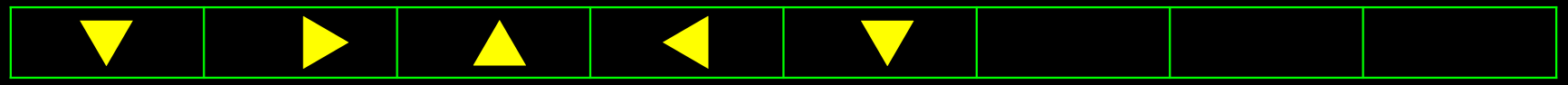
```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

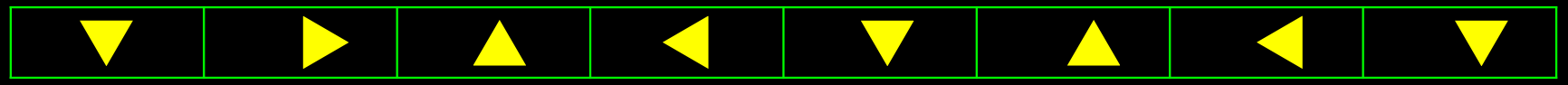
```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```



Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```

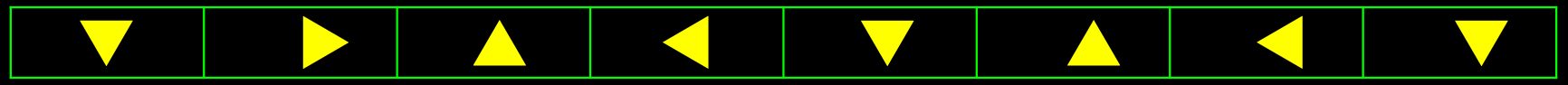


Produtores e consumidores

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        if(cont==N) sleep(cont);
        inserir(it);
        cont++;
        if(cont==1) wakeup(cont);
    }
}
```

```
Consumidor(){
    Item meu;
    while(true){
        if(cont==0) sleep(cont);
        meu = retirar();
        cont--;
        if(cont==N-1) wakeup(cont);
        consumir(meu);
    }
}
```

O PROBLEMA SAIU DA RC E FOI PARA *CONT*



Semáforos

Semáforos

- Proposta de Dijkstra (1965)

- Variáveis especiais para controlar o *número de sinais pendentes*

- Permitem apenas duas operações indivisíveis:
  - *Up* (originalmente V, *verhogen* → incrementar)
  - *Down* (originalmente P, *proberen* → testar)

Semáforos

- Oferecidos por meio de chamadas de sistema do SO
  - Operação em modo kernel
  - Interrupções desabilitadas

- Disponibilizados por meio de bibliotecas de programação
  - Operação pelo processo, de acordo com a compilação
  - Processo controla suas *threads* e pedidos

Semáforos

- Comportamento geral:
  - *Down*: tenta decrementar o contador. Se não for possível, manda o processo para bloqueio
  - Up: incrementa o contador e libera processos pendentes

Mutex

- Um *mutex* (*MUTual EXclusion*) é um semáforo simplificado: valores 0 ou 1 apenas
  - "Semáforos binários"

- Úteis em cenários mais simples, quando apenas o bloqueio de uma região é necessário

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
1

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
0

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
0

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
0

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
0

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
1

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
1

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
0

Semáforos (exemplo hipotético)

```
Debitar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo -=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

```
Creditar(Conta c, valor v)
   down(c.semaforo);
      saldo = c.saldo();
      saldo +=v;
      c.setSaldo(saldo);
   up(c.semaforo);
```

CONTA
semáforo
0

Produtor / consumidor com semáforos

- Precisamos proteger em três cenários:
  - *Buffer* cheio (produtor)
  - *Buffer* vazio (consumidor)
  - Escrita/retirada do *buffer* (ambos)

Produtor / consumidor com semáforos

- Precisamos proteger em três cenários:
  - *Buffer* cheio (produtor)
  - *Buffer* vazio (consumidor)
  - Escrita/retirada do *buffer* (ambos)

- Semáforo para posições cheias
- Semáforo para posições vazias
- Mutex para a escrita ou retirada

Produtor / consumidor com semáforos

```
int N = Buffer.CAPACIDADEMAXIMA;
Semaforo cheio = new Semaforo(0);
Semaforo vazio = new Semaforo(N);
Semaforo mutex = new Semaforo(1);
```

Produtor/consumidor com semáforos

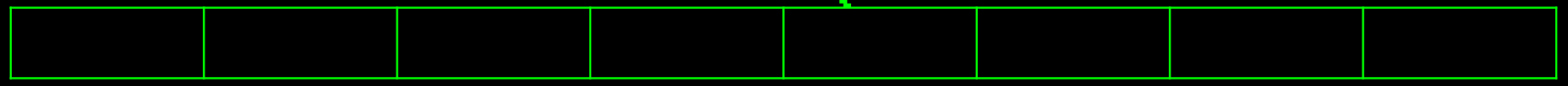
```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```

Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

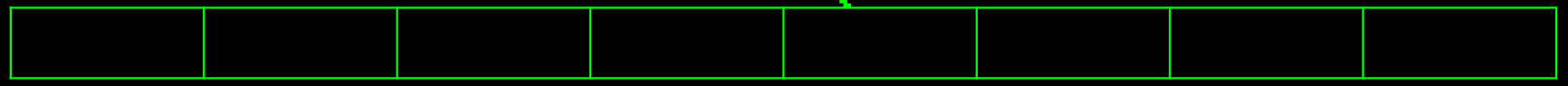
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

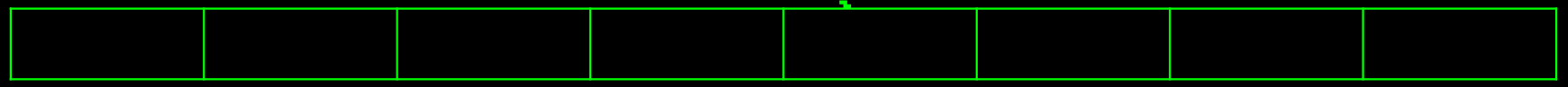
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

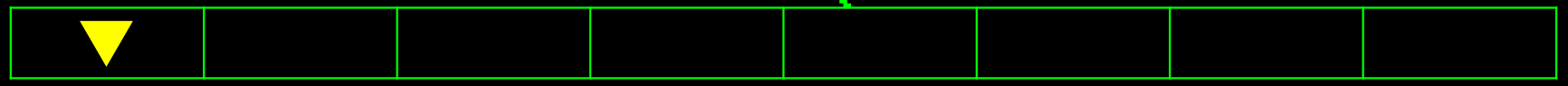
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

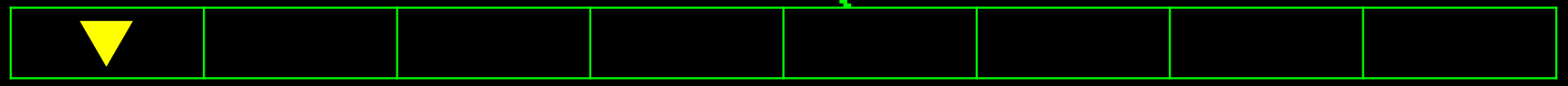
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

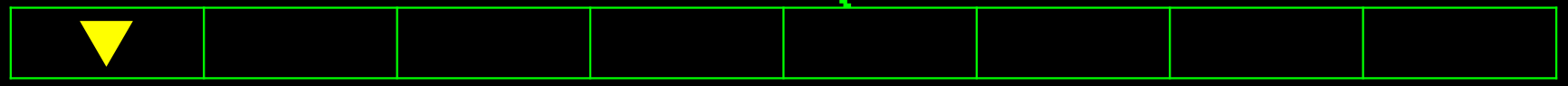
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

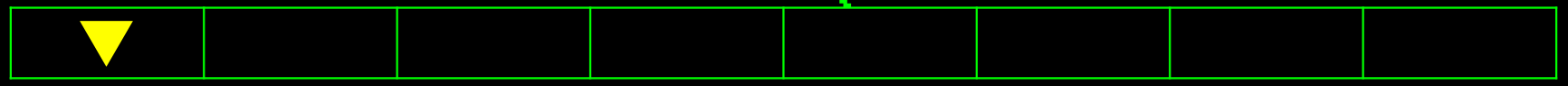
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

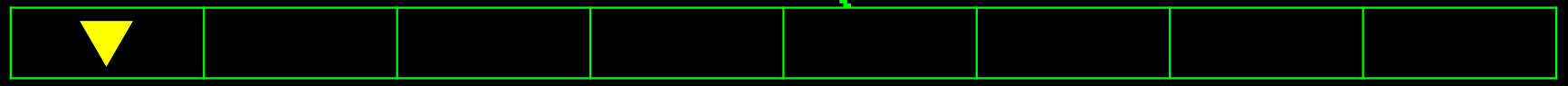
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

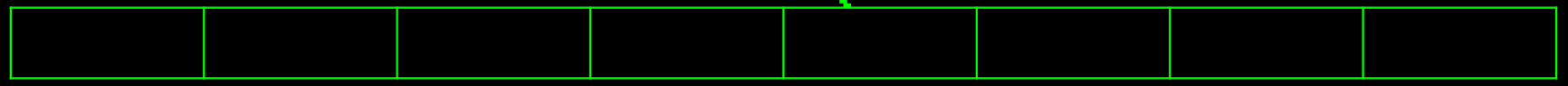
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

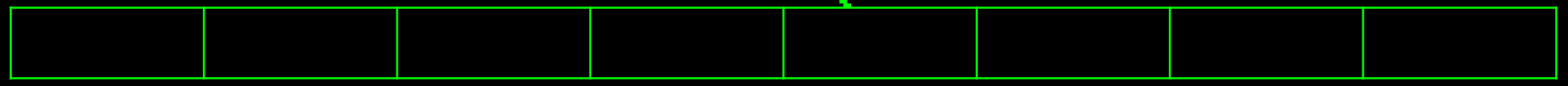
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

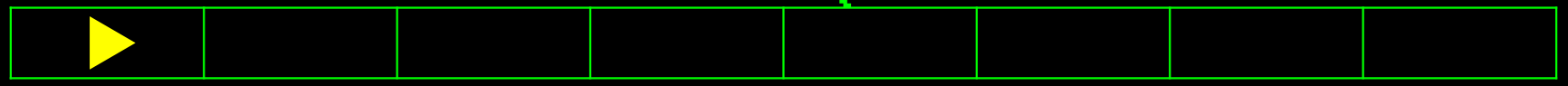
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

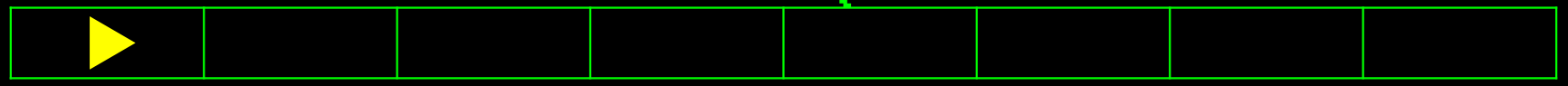
```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```

Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```

Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```

Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```

Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```

Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
      it = produzir();
      down(vazio);
      down(mutex)
         buf.inserir(it);
      up(mutex);
      up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
      down(cheio);
      down(mutex)
         it = buf.retirar(it);
      up(mutex);
      up(vazio);
      consumir(it);
   }
}
```

Semáforos

- Elegantes
- Eficientes

- Porém...
  - Programação deve ser realizada com extremo cuidado
    - A solução está com o programador, não no SO
  - *Tracing* e depuração de erros é muito difícil

SISTEMAS OPERACIONAIS

Exclusão mútua: Monitores

Prof. Matheus A. Souza
Eng. Software/Sist. Informação
PUC Minas

Semáforos e seus problemas

> Semáforos: solução elegante e eficiente

> Mas... necessita de programação cuidadosa e interfere diretamente na lógica do problema

Produtor/consumidor com semáforos

```
Produtor(){
   Item it;
   while(true){
   it = produzir();
   down(vazio)
   down(mutex);
   buf.inserir(it);
   up(mutex);
   up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
   down(cheio);
   down(mutex)
   it = buf.retirar(it);
   up(mutex);
   up(vazio);
   consumir(it);
   }
}
```

Produtor/consumidor com semáforos

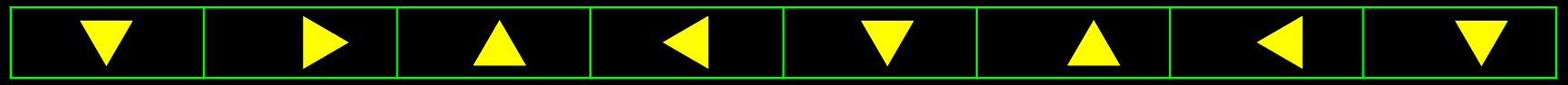
```
Produtor(){
   Item it;
   while(true){
   it = produzir();
   down(mutex)
   down(vazio);
   buf.inserir(it);
   up(mutex);
   up(cheio);
   }
}
```

```
Consumidor(){
   Item it;
   while(true){
   down(cheio);
   down(mutex)
   it = buf.retirar(it);
   up(mutex);
   up(vazio);
   consumir(it);
   }
}
```

Produtor/consumidor com semáforos

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        down(mutex)
        down(vazio);
            buf.inserir(it);
        up(mutex);
        up(cheio);
    }
}
```

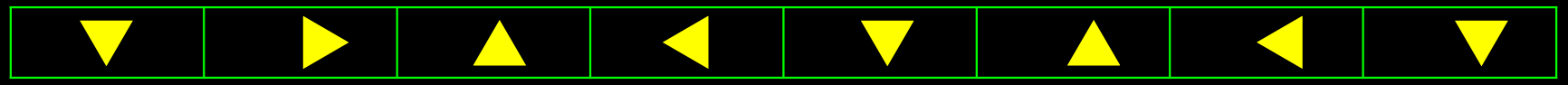
```
Consumidor(){
    Item it;
    while(true){
    down(cheio);
    down(mutex)
    it = buf.retirar(it);
    up(mutex);
    up(vazio);
    consumir(it);
    }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        down(mutex)
        down(vazio);
            buf.inserir(it);
        up(mutex);
        up(cheio);
    }
}
```

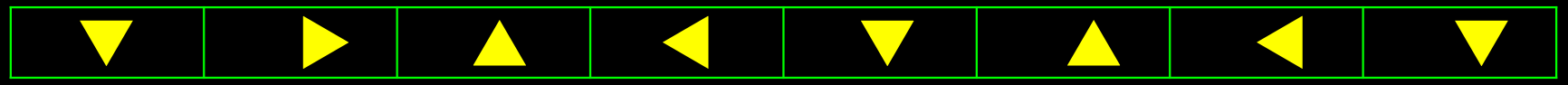
```
Consumidor(){
    Item it;
    while(true){
    down(cheio);
    down(mutex)
    it = buf.retirar(it);
    up(mutex);
    up(vazio);
    consumir(it);
    }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        down(mutex)
        down(vazio);
            buf.inserir(it);
        up(mutex);
        up(cheio);
    }
}
```

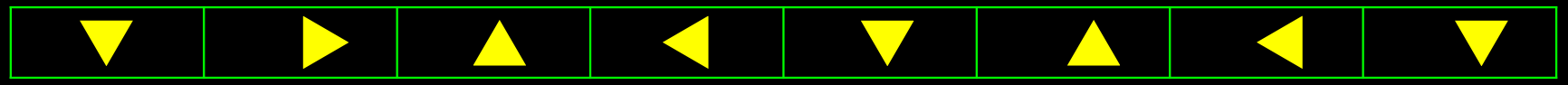
```
Consumidor(){
    Item it;
    while(true){
    down(cheio);
    down(mutex)
    it = buf.retirar(it);
    up(mutex);
    up(vazio);
    consumir(it);
    }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        down(mutex)
        down(vazio);
            buf.inserir(it);
        up(mutex);
        up(cheio);
    }
}
```

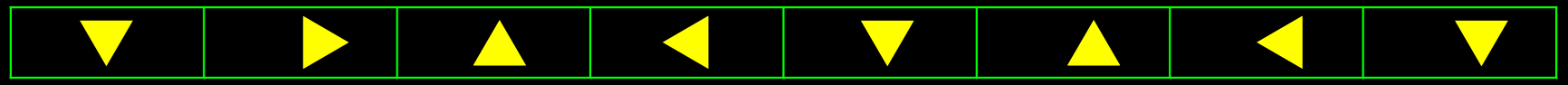
```
Consumidor(){
    Item it;
    while(true){
    down(cheio);
    down(mutex)
    it = buf.retirar(it);
    up(mutex);
    up(vazio);
    consumir(it);
    }
}
```



Produtor/consumidor com semáforos

```
Produtor(){
    Item it;
    while(true){
        it = produzir();
        down(mutex)
        down(vazio);
            buf.inserir(it);
        up(mutex);
        up(cheio);
    }
}
```

```
Consumidor(){
    Item it;
    while(true){
    down(cheio);
    down(mutex)
    it = buf.retirar(it);
    up(mutex);
    up(vazio);
    consumir(it);
    }
}
```



Monitores

- Propostas de Hansen(1973) e Hoare(1974)

- Solução de alto nível de abstração, provida pela linguagem

- Agrupamento unidades do programa (regiões críticas)
  - Dados e/ou métodos
  - Somente um processo pode estar ativo em um monitor em um instante de tempo

Monitores

- Esquema geral de Hansen (1973)
  - Variáveis de condição para controlar a espera
  - Dois sinais: *wait* e *signal*
  - *Signal* é sempre a última instrução do monitor

Monitores: produtor e consumidor

```
monitor ProducerConsumer
        condition full, empty;
        integer count;

        procedure insert(item: integer);
        begin
                if count = N then wait(full);
                insert_item(item);
                count := count + 1;
                if count = 1 then signal(empty)
        end;

        function remove: integer;
        begin
                if count = 0 then wait(empty);
                remove = remove_item;
                count := count − 1;
                if count = N − 1 then signal(full)
        end;

        count := 0;
end monitor;
```

```
procedure producer;
begin
        while true do
        begin
                item = produce_item;
                ProducerConsumer.insert(item)
        end
end;

procedure consumer;
begin
        while true do
        begin
                item = ProducerConsumer.remove;
                consume_item(item)
        end
end;
```

Monitores: produtor e consumidor

```
monitor ProducerConsumer
        condition full, empty;
        integer count;

        procedure insert(item: integer);
        begin
                if count = N then wait(full);
                insert_item(item);
                count := count + 1;
                if count = 1 then signal(empty)
        end;

        function remove: integer;
        begin
                if count = 0 then wait(empty);
                remove = remove_item;
                count := count − 1;
                if count = N − 1 then signal(full)
        end;

        count := 0;
end monitor;
```

```
procedure producer;
begin
        while true do
        begin
                item = produce_item;
                ProducerConsumer.insert(item)
        end
end;

procedure consumer;
begin
        while true do
        begin
                item = ProducerConsumer.remove;
                consume_item(item)
        end
end;
```

Monitores: produtor e consumidor

```
monitor ProducerConsumer
        condition full, empty;
        integer count;

        procedure insert(item: integer);
        begin
                if count = N then wait(full);
                insert_item(item);
                count := count + 1;
                if count = 1 then signal(empty)
        end;

        function remove: integer;
        begin
                if count = 0 then wait(empty);
                remove = remove_item;
                count := count − 1;
                if count = N − 1 then signal(full)
        end;

        count := 0;
end monitor;
```

```
procedure producer;
begin
        while true do
        begin
                item = produce_item;
                ProducerConsumer.insert(item)
        end
end;

procedure consumer;
begin
        while true do
        begin
                item = ProducerConsumer.remove;
                consume_item(item)
        end
end;
```

Monitores: produtor e consumidor

```
monitor ProducerConsumer
        condition full, empty;
        integer count;

        procedure insert(item: integer);
        begin
                if count = N then wait(full);
                insert_item(item);
                count := count + 1;
                if count = 1 then signal(empty)
        end;

        function remove: integer;
        begin
                if count = 0 then wait(empty);
                remove = remove_item;
                count := count − 1;
                if count = N − 1 then signal(full)
        end;

        count := 0;
end monitor;
```

```
procedure producer;
begin
        while true do
        begin
                item = produce_item;
                ProducerConsumer.insert(item)
        end
end;

procedure consumer;
begin
        while true do
        begin
                item = ProducerConsumer.remove;
                consume_item(item)
        end
end;
```

Monitores

- Em resumo: exclusão mútua “automática”
  - Solução simples
  - Menos probabilidades de erro
  - MAS dependem da linguagem de programação

Monitores

- Em resumo: exclusão mútua “automática”
  - Solução simples
  - Menos probabilidades de erro
  - MAS dependem da linguagem de programação

- Java, C#, ObjectPascal apresentam soluções próprias aproximadas ao conceito de monitor

Monitores: produtor e consumidor

```
public class ProducerConsumer{
   static final int N =100 // tamanho do buffer

   static monitor mon=new monitor();

   static producer p = new producer(mon);

   static consumer c = new consumer(mon);
```

Monitores: produtor e consumidor

```
class producer extends Thread{
   public void run(){
   int item;
   while(true){ // laco do produtor
   item=produce_item();
   mon.insert(item);
   }
   }
}

class consumer extends Thread{
   public void run(){
   int item;
   while(true){
   item=mon.remove();
   consume_item(item);
   }
   }
}
```

Monitores: produtor e consumidor

```
class monitor{
    private Buffer buffer = new Buffer(N);
    private int count=0;
    public synchronized void insert(int v){

        if(count==N) go_to_sleep();
        buffer.inserir(valor);
        count=count+1;
        if(count==1) notify(); //
    }
```

```
public synchronized int remove(){
    int val;
        if(count==0) go_to_sleep();
        val = buffer.retirar();
        count=count-1;
        if(count==N-1) notify();
        return val;
}


private void go_to_sleep(){

    try{wait();}
catch(InterruptedExceptionexc){};    }
```